

Algoritmi e Strutture Dati – Parte A – 04/09/2020

Esercizio A1, punti ≥ 8

Trovare i limiti superiori e inferiori più stretti possibili per la seguente famiglia di equazioni di ricorrenza, per valori di a interi positivi maggiori o uguali a 2:

$$T_a(n) = \begin{cases} 2T(\lfloor n/a \rfloor) + n^{\lceil a/2 \rceil} & n \geq 2 \\ 1 & n < 2 \end{cases}$$

Esercizio A2 – Punti ≥ 10

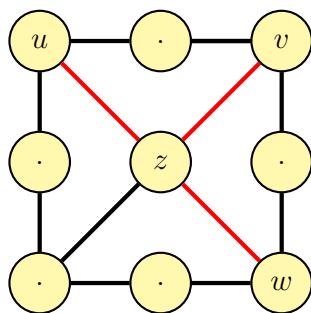
Sia G un grafo non orientato e connesso che rappresenta una rete stradale e siano u, v, w tre nodi (città) in questa rete. Tre amici che abitano in questi tre nodi vogliono incontrarsi in un quarto nodo z , in modo tale che la distanza che devono percorrere (misurata come somma del numero minimo di archi che ognuno di essi deve attraversare per partire da u, v, w e raggiungere z) sia la minore possibile. Nel caso più nodi abbiano distanza minore, andrà bene uno qualunque di essi. Scrivere un algoritmo

NODE meetingPoint(GRAPH G , NODE u , NODE v , NODE w)

che prenda in input G, u, v, w e restituisca il nodo z .

Discutere correttezza e complessità computazionale dell'algoritmo proposto.

Nel grafo seguente, il nodo z è quello al centro; la distanza totale di u, v, w è pari a $1 + 1 + 1 = 3$. Scegliere v dà origine ad una distanza totale pari $2 + 0 + 2 = 4$, da tutti gli altri nodi la distanza è superiore a 4.



Notate che è possibile che la città scelta sia una di quelle di partenza, se questo minimizza il numero di archi da percorrere. Per esempio, se nel grafo precedente gli amici abitassero nelle città u, z, w , la città prescelta sarebbe di nuovo z .

Esercizio A3 – Punti ≥ 12

Scrivere un algoritmo

`int longestSingle(int[] A, int n)`

che prenda in input un vettore A di dimensione n , contenente interi compresi fra 1 ed n , e restituisca la lunghezza del più lungo sottovettore contiguo che non contiene valori duplicati.

Ad esempio, nel sottovettore $[1, 2, \mathbf{3}, \mathbf{2}, \mathbf{4}, 1, 3]$ di lunghezza $n = 7$, i sottovettori $[3, 2, 4, 1]$ e $[2, 4, 1, 3]$ hanno lunghezza massima. L'algoritmo dovrà restituire 4.

Discutere correttezza e complessità computazionale dell'algoritmo proposto.